



EVERYONE READS THIS FIRST

Campus Placement and Training Cell

Advanced Java project, B.Sc. IT Semester V. Ten minutes here and you can explain the whole app.

2

portals: student,
officer

25

students

10

companies

14

drives

85

applications

146

automated tests

1. What the app is

SAY THIS

A college placement cell has to tell hundreds of students which company drives they can sit for, collect their applications, and move each one through shortlist, interview and offer. This app does that in one place. A student sees only the drives their own record qualifies them for, and when they do not qualify it says exactly why. The placement officer announces drives, tracks every candidate and reads live reports straight from the database.

It is a Jakarta EE 10 web application: Servlets and JSP on the front, EJB beans for the business rules, JPA with Hibernate and hand-written JDBC underneath, and MySQL 8 as the database.

2. Who exactly uses it

USER	HOW THEY GET AN ACCOUNT	WHAT THEY CAN DO	THEIR PAGES
Student	Registers themselves at <code>/register</code> , or the officer imports them in bulk.	Keep a profile and resume, browse drives, see eligibility with reasons, apply, withdraw, follow status and notes from the cell.	<code>/app/ ...</code>
Placement officer (admin)	Created by the college, never through the public form. Three exist in the demo data.	Add companies, announce and close drives, move candidates through the pipeline, shortlist in batches, import students, read reports, browse the database, read the audit trail.	<code>/admin/ ...</code>

What about the companies?

Recruiters do not log in. A company is a record the officer maintains, and the officer posts drives on its behalf, which is how a placement cell really works: the cell is the go-between. If the teacher asks "who puts up the job opportunities", the answer is the placement officer, from the Drives page.

3. How data gets in, and how it gets out

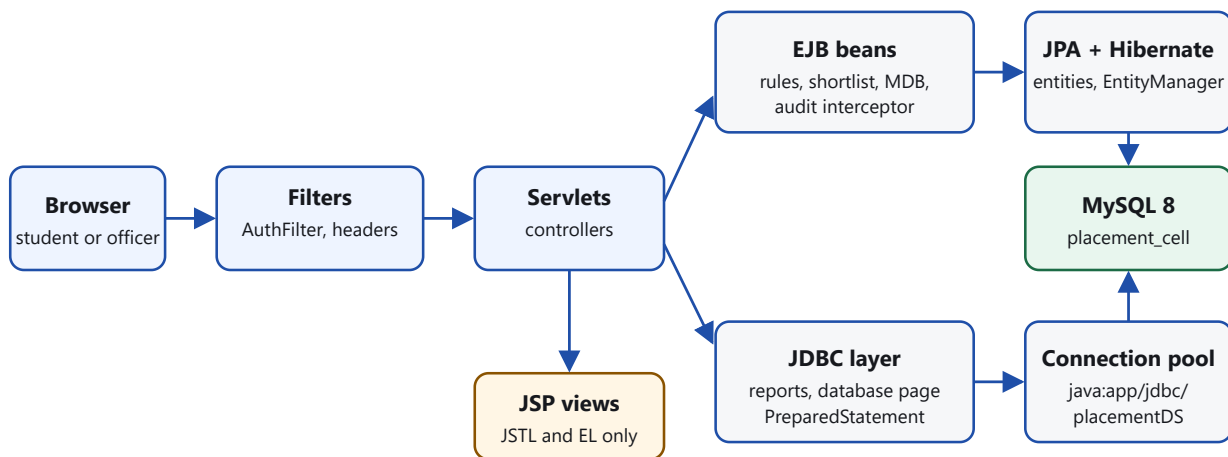
Data in

WHO	ACTION	LANDS IN TABLE
Student	Register	<code>app_user</code> , <code>student_profile</code>
Student	Edit profile, upload resume	<code>student_profile</code> and a file on disk
Student	Apply or withdraw	<code>job_application</code>
Officer	Add a company	<code>company</code>
Officer	Announce or close a drive	<code>drive</code>
Officer	Bulk import students	<code>app_user</code> , <code>student_profile</code>
Officer	Move a candidate, shortlist	<code>job_application</code> status and note
System	Every business write	<code>audit_log</code>
System	First start on an empty database	all tables, from <code>DataSeeder</code>

Data out

WHO	SCREEN	WHAT THEY SEE
Student	Drives board	Open drives, eligible or not, with the reason
Student	My applications	Status and the cell's latest note
Student	Resume link	Their own file, downloaded by the server
Officer	Overview	Totals, placed percentage, recent applications
Officer	Pipeline	Every application, filtered by drive and status, with a control to move it along
Officer	Reports	Branch and company numbers from plain JDBC, with the SQL shown
Officer	Database	Schema, rows and a read only SQL console
Officer	Audit	Who did what, how long it took
Both	Notification	A JMS message is queued on apply and on status change

4. Architecture in one picture



A servlet never writes HTML itself: it forwards to a JSP under WEB-INF, so no view can be opened by URL. Both data paths share one pool.

Read it left to right. A request meets the filters, then a servlet. The servlet asks an EJB (business rules and normal reads and writes through JPA) or the JDBC layer (reports and the database page). Both end at the same MySQL through the same server managed pool. The result goes back to a JSP, which produces the page.

5. Features at a glance

Student portal

- Self registration with a real email domain check
- Sign in with lockout after repeated failures
- Profile: branch, CGPA, backlogs, phone, resume upload
- Drives board that hides what you cannot apply for and says why
- Apply, withdraw, and follow status with the cell's notes
- A welcome panel after sign up and a getting started checklist on the overview
- Light and dark theme, remembered in a cookie

Officer portal

- Companies and drives with eligibility rules per drive
- Application pipeline and a batch shortlist
- Bulk import of students
- Reports read with plain JDBC, SQL shown beside the result
- Database page: schema, rows, guarded SQL console
- Audit trail, system page, and the Classroom JDBC page

6. Which course topic lives where

TOPIC FROM THE COURSE	WHERE IT IS IN THIS APP
Servlets, lifecycle, annotations	Every class in <code>web/</code> , declared with <code>@WebServlet</code>

RequestDispatcher	<code>Web.render</code> forwards every page to its JSP
Cookies	Remember-my-email, theme, last visit
HttpSession and listener	Signed-in user in the session, <code>SessionTracker</code> counts sessions
File upload and download	Resume upload, <code>ResumeDownloadServlet</code>
Non blocking I/O	<code>BulkImportServlet</code> reads its body with a <code>ReadListener</code>
JSP, actions, EL, JSTL	All views, scriptlets switched off in <code>web.xml</code>
EJB: stateless, stateful, singleton, MDB, interceptor	Services, <code>ShortlistBean</code> and <code>MockScoreBean</code> , <code>DataSeeder</code> , <code>NotificationMDB</code> , <code>AuditInterceptor</code>
JNDI	The datasource name and the stateful bean lookups
JPA with Hibernate	<code>entity/</code> and <code>persistence.xml</code>
JDBC	<code>jdbc/</code> , plus the classroom style <code>MockScoreBean</code>

7. Who explains what

PERSON	AREA	READ
Karan	The app itself: users, data in and out, demo, seed data, the student and officer flows	<code>01-karan-app-overview</code>
Yuvraj	MySQL connectivity and JDBC, the connectivity demo	<code>02-yuvraj-mysql-jdbc</code>
Tiya	Account creation, login, passwords, sessions, security	<code>03-tiya-auth-security</code>
Kinjal	JSP views, validation, resume upload, bulk import	<code>04-kinjal-views-validation</code>
Chandra	Entities, JPA, Hibernate, every EJB, tests	<code>05-chandra-entities-ejb</code>

Everyone also reads `06-viva-questions` (every question we expect, with an owner and an answer) and `07-demo-run-sheet` (the order of the presentation, with timings).

8. Logins for the demo

ROLE	EMAIL	PASSWORD	USE IT FOR
Officer	<code>tpo@campus.edu</code>	<code>Campus@2026</code>	The officer walkthrough
Student	<code>aarti.deshpande@campus.edu</code>	<code>Campus@2026</code>	Main student demo: eight applications, no offer yet, drives left to apply for
Student	<code>rahul.menon@campus.edu</code>	<code>Campus@2026</code>	A placed student (has an offer)
Student	<code>sneha.pillai@campus.edu</code>	<code>Campus@2026</code>	CGPA 9.1, a second live account

Every seeded account shares one password. That is for the demo only, and the hashes in the database are all different because each has its own salt.

9. Get it running on your laptop

1. Clone the repository and open the folder `placement-cell` in VS Code.
2. Start the MySQL 8 service. In MySQL Workbench, create the database and user once (the four lines are in `README.md`).
3. In PowerShell, once per machine: `.\setup.ps1` (downloads the server and JDK into `server\`).
4. Every time: `.\run.ps1`. On the morning of the presentation use `.\run.ps1 -Reseed` so every drive has a fresh deadline.
5. Open `http://localhost:8080/placement/`. Stop with `.\stop.ps1`.
6. Shortcut: double click `start.bat` in the folder. It builds, starts the server, deploys and opens the browser.

CAREFUL

The seeded dates are counted from the day the data is created. If the database was created days ago, drives can show "the last date to apply has passed". Run `.\run.ps1 -Reseed` to fix that.